

Informe técnico: Customer Churn Prediction

MCD202 - Machine Learning

Avila Edgar, Condori Rodrigo, Mamani Isaac

09/12/2026

Entrenamiento del modelo

1. Fuente de datos y carga inicial

El dataset está en `data/customer_churn_dataset-testing-master.csv`. Tiene 64 374 filas y 12 columnas. Cada fila describe a un cliente con diez características y la variable objetivo `Churn`. El valor 1 indica abandono.

`ml/seed.py` carga las primeras 20 000 filas del CSV a la base SQLite `db/churn.db`. El script borra la tabla `customers` y la vuelve a llenar. Así los datos de entrenamiento siempre reflejan la última fuente antes de una ejecución.

2. Ingreso incremental de clientes

`ml/ingest.py` simula el crecimiento de la base. Cada 5 minutos inserta la primera fila del CSV que todavía no está en la base, comparando por `CustomerID`. Cuando el CSV se agota, el proceso sigue vivo pero no inserta nada.

3. Características y preprocesamiento

`ml/features.py` fija el contrato de columnas. Siete son numéricas y tres son categóricas:

Tipo	Columnas
Numéricas	Age, Tenure, Usage Frequency, Support Calls, Payment Delay, Total Spend, Last Interaction
Categóricas	Gender, Subscription Type, Contract Length

`train.py` arma el preprocesamiento con `ColumnTransformer` y lo deja dentro del pipeline:

- `StandardScaler` para las columnas numéricas.
- `OneHotEncoder(handle_unknown="ignore")` para las categóricas.

`handle_unknown="ignore"` es deliberado. Si en producción llega una categoría que el modelo nunca vio, la codificación no rompe la inferencia.

`train.py` ajusta el pipeline completo y lo guarda con `joblib`. La API no repite el encoding por su cuenta; lo hereda del artefacto. Por eso la transformación en producción coincide con la de entrenamiento.

4. Algoritmo y partición de datos

El clasificador es `RandomForestClassifier` con estos parámetros:

Parámetro	Valor
<code>n_estimators</code>	200
<code>max_depth</code>	12
<code>min_samples_leaf</code>	5
<code>n_jobs</code>	-1
<code>random_state</code>	42

Elegimos RandomForest por una razón práctica: da resultados sólidos en tablas de este tamaño y se entrena sin GPU. Los fijamos probando sobre la partición de entrenamiento; no hay una búsqueda exhaustiva detrás.

`train.py` divide los datos con `train_test_split`: 80% para entrenar y 20% para probar, con `random_state=42` y `stratify=y`. El estratificado conserva la proporción de churn en los dos conjuntos.

5. Métricas y resultados

`train.py` calcula dos métricas sobre el conjunto de prueba:

- Accuracy, con umbral de 0.5 sobre la probabilidad predicha.
- ROC AUC, sobre la probabilidad directa.

En la prueba más reciente se entrenó con 16 004 filas. Dio accuracy de 0.9945 y ROC AUC de 0.99995.

Al principio se probó con menos filas, 1 600, y los resultados fueron más pobres: accuracy de 0.955. El salto llegó al pasar a 16 000 filas. Pero esas métricas miden el conjunto de prueba, que sale de la misma fuente que el entrenamiento. No hay validación cruzada ni análisis de errores por clase; queda como trabajo pendiente.

6. Versionado de artefactos

Cada ejecución que ve datos nuevos genera una versión nueva. La estructura por versión es:

- `model/<version>/model.pkl` — el pipeline serializado.

- `model/<version>/manifest.json` — hash de datos, filas, fecha y métricas.
- `model/current.txt` — puntero a la versión vigente.

Antes de entrenar, `train.py` calcula un hash md5 de las filas (características más objetivo). Si el hash coincide con el del último manifiesto, el entrenamiento se omite y se conserva la versión vigente. Eso ocurre cuando `ingest.py` ya no encuentra filas nuevas en el CSV.

La numeración de versiones es propia de cada entorno. El historial generado durante las pruebas no se arrastra a producción; ahí el conteo comienza de nuevo.

7. Reentrenamiento programado

`scheduler.py` corre en un ciclo de 12 horas (43 200 segundos). Cada ciclo ejecuta `train.py` y después `predict.py`. El primer ciclo corre al arrancar, de modo que la aplicación tiene modelo entrenado y predicciones desde el inicio.

El servicio `scheduler` del `docker-compose.yml` lo lanza así:

```
1 python scheduler.py --db /db/churn.db --model-dir /opt/model --interval 43200
```

`predict.py` carga el modelo activo, calcula `predict_proba` para todos los clientes y guarda los resultados en la tabla `predictions`. Las predicciones se conservan por versión de modelo, así el historial de churn de cada cliente se puede rastrear entre versiones.

Aplicación

1. Arquitectura de la aplicación

La aplicación entrega predicciones individuales de abandono de clientes mediante una API REST construida con FastAPI y una interfaz web estática servida por la misma aplicación. La responsabilidad de esta capa es validar el contrato de entrada, convertirlo al formato de inferencia y presentar el resultado; no entrena ni reentrena el modelo.

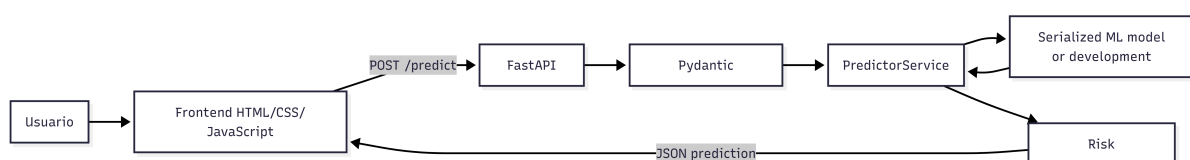


Figure 1: Diagrama de arquitectura

La estructura bajo `/app` separa responsabilidades:

Ruta	Responsabilidad
<code>main.py</code>	Punto de entrada ASGI. Construye la instancia FastAPI, registra rutas, manejadores globales de errores y publica el frontend.
<code>api/</code>	Contratos Pydantic en <code>schemas.py</code> y definición HTTP de los endpoints en <code>routes.py</code> . Los endpoints delegan la inferencia al servicio.
<code>services/</code>	Lógica de negocio independiente del transporte HTTP: carga e inferencia del modelo (<code>predictor.py</code>) y clasificación de riesgo (<code>risk.py</code>).
<code>core/</code>	Configuración mediante variables de entorno (<code>config.py</code>) y excepciones/manejadores de errores comunes (<code>exceptions.py</code>).
<code>frontend/</code>	Interfaz estática: formulario de cliente, llamada <code>fetch</code> al endpoint y tabla de predicciones realizadas durante la sesión del navegador.
<code>tests/</code>	Pruebas unitarias y de endpoint. Usan un modelo mock, por lo que no requieren el artefacto real del equipo de ML.
<code>Dockerfile</code>	Definición de una imagen ejecutable de la aplicación.

2. Endpoints de la API

GET `/health`

Comprueba la disponibilidad de la aplicación y del servicio de inferencia. No recibe cuerpo de petición.

Respuesta exitosa (200 OK):

```
1 {  
2   "status": "ok",
```

```
3  "model_loaded": true,  
4  "model_source": "artifact"  
5  }
```

`model_source` puede ser `artifact` si se cargó el archivo configurado, `fallback` si está activo el modelo local de demostración, o `injected` cuando se inyecta un modelo en pruebas. El estado es `ok` mientras exista un modelo disponible; de lo contrario sería `degraded`.

POST /predict

Valida un cliente, llama al modelo y devuelve la probabilidad de churn junto con una banda de riesgo. Recibe JSON:

```
1  {  
2    "CustomerID": "C-100",  
3    "Age": 35,  
4    "Gender": "Female",  
5    "Tenure": 12,  
6    "Usage Frequency": 15,  
7    "Support Calls": 2,  
8    "Payment Delay": 3,  
9    "Subscription Type": "Premium",  
10   "Contract Length": "Annual",  
11   "Total Spend": 650.5,  
12   "Last Interaction": 4  
13 }
```

Respuesta exitosa (200 OK):

```
1  {  
2    "customer_id": "C-100",  
3    "churn_probability": 0.78,  
4    "risk_level": "High"  
5  }
```

La probabilidad se redondea a cuatro decimales en la respuesta. La clasificación implementada es: `Low` para valores menores de 0.35, `Medium` desde 0.35 hasta menos de 0.65 y `High` desde 0.65.

GET /customers

Es un endpoint reservado para una futura lista de predicciones o clientes persistidos. Actualmente no recibe parámetros y devuelve siempre una lista vacía:

```
1  []
```

Pendiente / trabajo futuro: no existe todavía una base de datos, repositorio ni persistencia de clientes. La tabla del frontend conserva resultados solamente en el DOM de la sesión actual.

3. Integración con el modelo de ML

La clase `PredictorService` lee `MODEL_PATH` y carga el artefacto mediante `joblib.load()`. El objeto debe exponer `predict_proba(features)` y devolver una estructura binaria donde la columna/posición `[0] [1]` es la probabilidad de churn. Se recomienda entregar un pipeline serializado que incluya el preprocesamiento, para que encoding y transformaciones se apliquen de la misma forma que durante el entrenamiento.

Antes de inferir, el servicio elimina `CustomerID` y crea un `pandas.DataFrame` de una fila. Por defecto, las columnas enviadas al modelo son:

```
1 Age, Gender, Tenure, Usage Frequency, Support Calls, Payment Delay,  
2 Subscription Type, Contract Length, Total Spend, Last Interaction
```

La ruta opcional `FEATURE_LIST_PATH` permite suministrar el orden/nombre de columnas como una lista JSON o como `{"features": [...]}`. Si el contrato del modelo cambia, se debe actualizar conjuntamente esa lista y los alias/validaciones de `api/schemas.py`.

Variables relevantes:

Variable	Comportamiento
<code>MODEL_PATH</code>	Ubicación del archivo <code>joblib/pickle</code> . Localmente el valor por defecto apunta a <code>../model/model.pkl</code> respecto a la carpeta <code>app</code> .
<code>ALLOW_FALLBACK_MODEL</code>	Si es <code>true</code> y no se puede cargar el artefacto, activa <code>FallbackChurnModel</code> , un modelo heurístico determinista para desarrollo.
<code>FEATURE_LIST_PATH</code>	Ruta opcional al archivo JSON con el contrato de columnas.

Con `ALLOW_FALLBACK_MODEL=false`, un archivo ausente, ilegible o inválido impide inicializar el predictor y se genera un error de modelo no disponible. El fallback no es un modelo entrenado y no debe usarse para decisiones de producción. El Dockerfile lo desactiva de forma predeterminada.

4. Validación y manejo de errores

Pydantic valida el cuerpo de `POST /predict` antes de llamar al modelo. Los campos requeridos son todos los del ejemplo de cliente. Las principales reglas son:

Campo	Regla implementada
<code>CustomerID</code>	Texto obligatorio, de 1 a 128 caracteres.
<code>Age</code>	Entero entre 18 y 120.
<code>Gender</code>	Exactamente <code>Male</code> o <code>Female</code> .
<code>Tenure</code>	Entero entre 0 y 1200.
<code>Usage Frequency, Support Calls</code>	Enteros entre 0 y 10000.
<code>Payment Delay, Last Interaction</code>	Enteros entre 0 y 3650.
<code>Subscription Type, Contract Length</code>	Texto obligatorio; tras quitar espacios no puede estar vacío y tiene máximo 100 caracteres.
<code>Total Spend</code>	Número entre 0 y 10 000 000.

Los manejadores globales devuelven JSON uniforme:

Código	Significado
200	Petición procesada correctamente.
422	Falló la validación Pydantic. La respuesta contiene <code>detail: "Input validation failed"</code> y una lista <code>errors</code> con los campos y reglas incumplidos.
500	No hay modelo disponible, el modelo no produjo una predicción válida o ocurrió un error interno inesperado. Se devuelve <code>detail</code> con una descripción segura del problema.

La aplicación usa el módulo estándar `logging` para registrar errores de modelo, errores de inferencia y errores inesperados; no usa `print`.

5. Pruebas de funcionamiento

La suite se ejecuta desde `/app`:

```
1 python -m pytest -q
```

Los casos actuales son:

Archivo	Cobertura
<code>test_predict_endpoint.py</code>	<code>POST /predict</code> con cliente válido (200 y riesgo <code>High</code> con el mock) y con edad inválida (422 y mensaje de validación).
<code>test_health_endpoint.py</code>	<code>GET /health</code> informa estado <code>ok</code> , modelo cargado y fuente <code>injected</code> .
<code>test_predictor_service.py</code>	<code>PredictorService</code> crea el <code>DataFrame</code> y consume correctamente <code>predict_proba</code> del modelo mock.
<code>test_risk_classification.py</code>	Bandas Low/Medium/High, casos límite 0.35 y 0.65, y rechazo de probabilidad fuera de [0, 1].
<code>conftest.py</code>	Cliente FastAPI, cliente de ejemplo y modelo mock reutilizables.

El resultado esperado es una suite exitosa sin depender de `model.pkl`. En la versión implementada se obtienen 11 pruebas aprobadas.

6. Despliegue con Docker

El `Dockerfile`:

1. Parte de `python:3.11-slim`.
2. Define variables de ejecución: salida Python sin búfer, `PORT=8000`, `MODEL_PATH=/opt/model/model.pkl` y fallback desactivado.
3. Establece `/opt/app` como directorio de trabajo.
4. Copia e instala `requirements.txt` sin caché de pip.
5. Copia el código de `/app`, expone el puerto 8000 y ejecuta Unicorn enlazado a `0.0.0.0`.

Desde la raíz del repositorio:

```
1 docker build -t churn-risk-app -f app/Dockerfile app
2 docker run --rm -p 8000:8000 -e MODEL_PATH=/opt/model/model.pkl -v "${PWD}/model:/opt/model:ro" churn-risk-app
```

El volumen monta el artefacto en modo solo lectura. Para producción se debe conservar `ALLOW_FALLBACK_MODEL=false`, especificar una ruta de modelo accesible y definir, si corresponde, `FEATURE_LIST_PATH`. `PORT` modifica el puerto de Uvicorn dentro del contenedor; si se modifica, también debe ajustarse el mapeo `-p`.

7. Guía de inicio para un integrante nuevo

1. Ubicarse en la carpeta de aplicación:

```
1 cd "D:\PROYECTO MAESTRIA\trabajo-machine-learning\app"
```

2. Crear y activar un entorno virtual:

```
1 python -m venv .env
2 .env\Scripts\Activate.ps1
```

3. Instalar dependencias y crear configuración local:

```
1 pip install -r requirements.txt
2 Copy-Item .env.example .env
```

4. Para desarrollo sin artefacto ML, mantener `ALLOW_FALLBACK_MODEL=true` en `.env`. Para usar el modelo real, establecer `MODEL_PATH` con su ruta y preferiblemente definir `ALLOW_FALLBACK_MODEL=false`.

5. Ejecutar pruebas:

```
1 python -m pytest -q
```

6. Iniciar el servidor:

```
1 uvicorn main:app --reload --port 8000
```

7. Abrir <http://localhost:8000> para la interfaz o <http://localhost:8000/docs> para probar la API mediante OpenAPI.

Antes de conectar el artefacto ml, el equipo debe confirmar el orden exacto de features, categorías válidas y que la clase positiva de `predict_proba` corresponde realmente a churn.

Despliegue de la aplicación

1. Composición de servicios

Toda la aplicación corre con Docker Compose. El archivo `docker-compose.yml` define cinco servicios:

Servicio	Rol
<code>db_seed</code>	Carga el CSV en SQLite una sola vez. Toma las primeras 20 000 filas.
<code>ingest</code>	Inserta un cliente nuevo cada 5 minutos, simulando el crecimiento de la base.
<code>scheduler</code>	Entrena cada 12 horas y predice en lote sobre todos los clientes.
<code>app</code>	API FastAPI y frontend. Escucha en el puerto 8000 dentro de la red interna.
<code>caddy</code>	Reverse proxy con HTTPS automático.

Las dependencias están ordenadas. `db_seed` termina primero, de ahí dependen `ingest` y `scheduler`, y `app` arranca cuando el `scheduler` ya está corriendo. Caddy queda delante de todos.

`app`, `scheduler` e `ingest` montan los mismos volúmenes: `./model` y `./db`. Todos ven el mismo artefacto y los mismos datos.

2. Caddy como reverse proxy

El `Caddyfile` es corto:

```
1 {$DOMAIN} {
2     encode zstd gzip
3     reverse_proxy app:8000
4 }
```

Caddy escucha en los puertos 80 y 443. Genera el certificado TLS por su cuenta con Let's Encrypt, comprime las respuestas con zstd o gzip y reenvía el tráfico al contenedor `app`. El dominio se pasa por la variable `CADDY_DOMAIN`.

El puerto 8000 de la aplicación no se publica hacia afuera. Solo Caddy queda expuesto; el resto de servicios vive en la red interna de Compose. `CADDY_LISTEN` controla la interfaz de escucha y por defecto apunta a `127.0.0.1` para pruebas locales.

3. Servidor y dominio

El despliegue corre en un VPS de Hetzner con Docker. El dominio se registró en no-ip. El hostname se pasa a Compose con `CADDY_DOMAIN` y Caddy lo usa para emitir el certificado y responder por HTTPS. La aplicación queda accesible en <https://customer-churn.ddns.net/>.

Pruebas de funcionamiento

La suite automatizada con pytest se describe en la sección Aplicación, punto 5. Aparte de eso, se probó el sistema completo ya desplegado contra la API y la interfaz web:

- `GET /health` responde 200 con estado `ok` y el modelo cargado desde el artefacto.
- La interfaz web recibe el formulario, llama al endpoint y muestra la predicción en la tabla de la sesión.

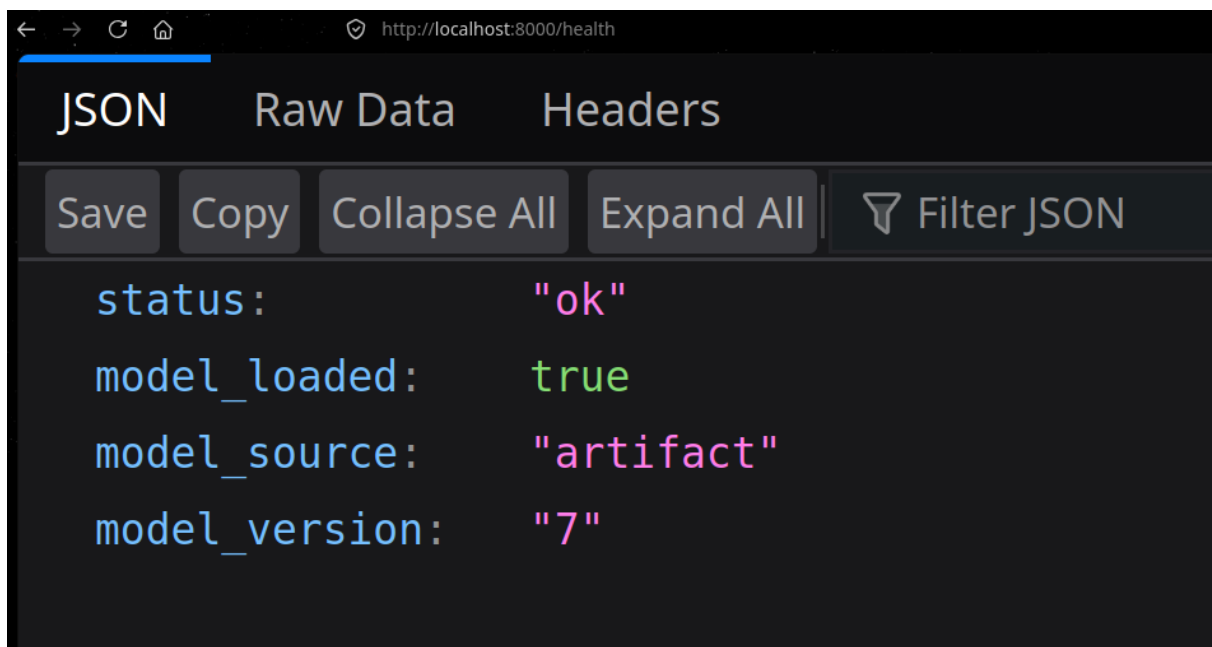


Figure 2: Prueba del endpoint de salud

Riesgo de abandono de clientes

Estime la probabilidad de que un cliente abandone su suscripción.

Customer ID	Age	Gender	Tenure (months)
		Male	
Usage Frequency	Support Calls	Payment Delay (days)	Subscription Type
Contract Length	Total Spend	Last Interaction (days)	
			Predecir riesgo

[Ver todos los clientes con su churn](#)

Clientes predichos

Customer	Churn probability	Risk
----------	-------------------	------

Figure 3: Prueba de la interfaz web

Cientes y riesgo de abandono

Página 1 de 2015 · 20145 clientes · modelo v2

Customer	Edad	Género	Suscripción	Contrato	Churn	Riesgo	Modelo	
4781	62	Female	Standard	Monthly	99.1%	High	v2	
13575	52	Female	Premium	Monthly	98.9%	High	v2	
15302	52	Female	Standard	Monthly	98.9%	High	v2	
5498	56	Female	Standard	Monthly	98.8%	High	v2	
16508	55	Female	Standard	Quarterly	98.3%	High	v2	
11694	52	Female	Premium	Annual	98.1%	High	v2	
14873	61	Female	Standard	Monthly	98.1%	High	v2	
17026	58	Female	Standard	Monthly	97.9%	High	v2	
12009	65	Female	Standard	Monthly	97.9%	High	v2	
9662	24	Female	Standard	Monthly	97.8%	High	v2	

Figure 4: Tabla de customers con churn

Dashboard



Figure 5: Dashboard